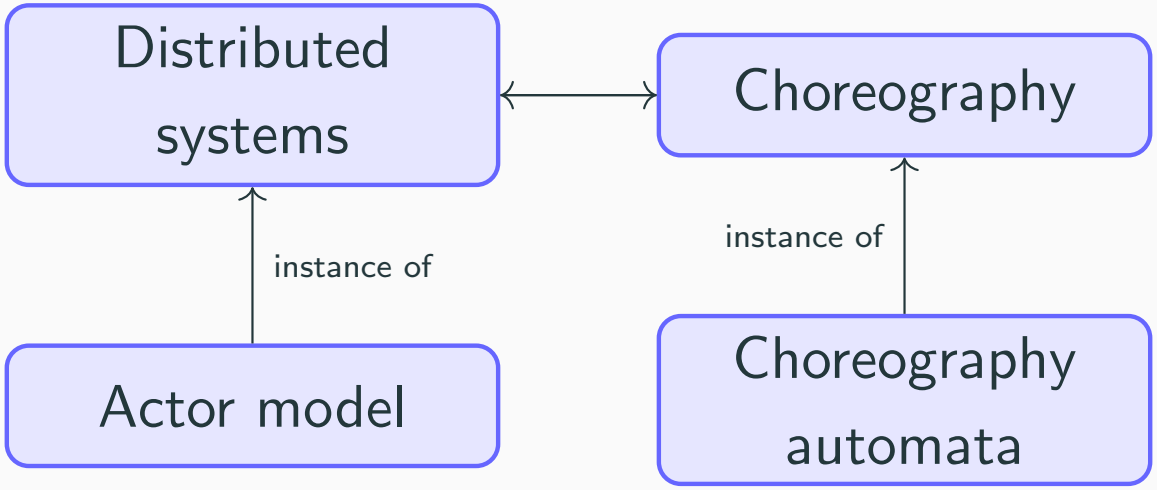
Choreographies for Program Understanding

Gabriele Genovese, Ivan Lanese, Cinzia Di Giusto, Emilio Tuosto, and Germán Vidal

17 June 2025

Distributed Systems are Hard

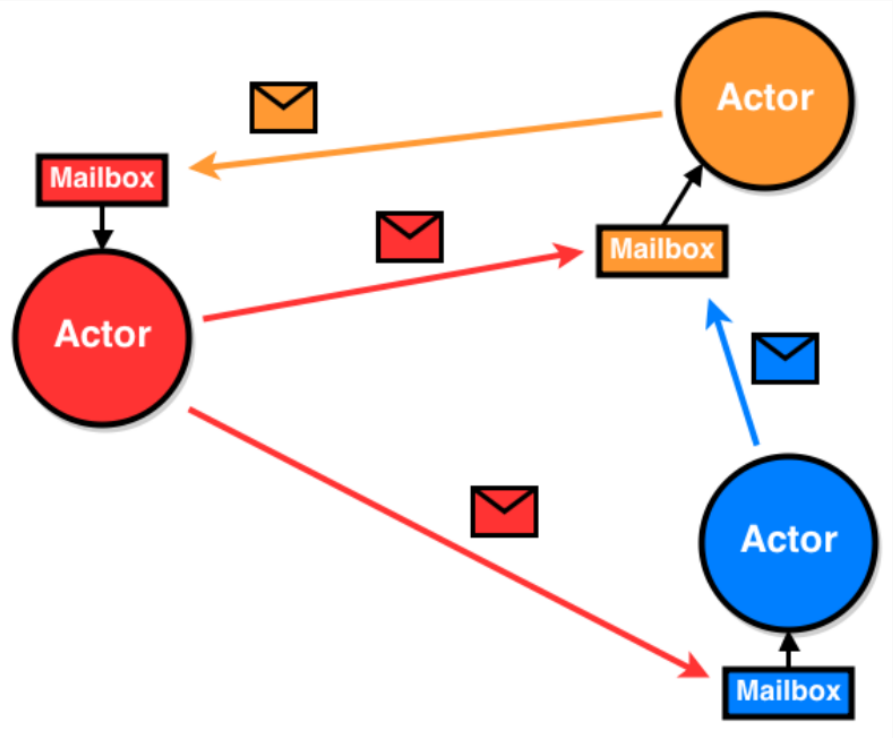
Abstractions in order to simplify:



The Actor Model

Main concepts

- Processes with mailbox
- Asynchronous messaging



The Actor Model

Ecosystem

- Erlang, Elixir, Scala
- Akka (Java), Actix (Rust)

Choreography Automata [1]

Informally

- Choreography: describes distributed protocols
- Paired with automata theory

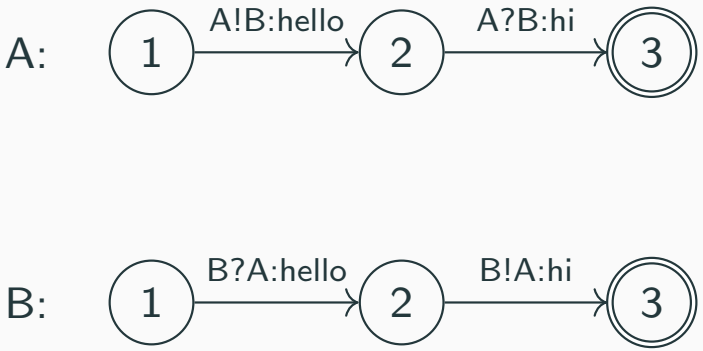
[1] Barbanera et al. "Choreography automata." COORDINATION, 2020.

Global vs Local View

Global

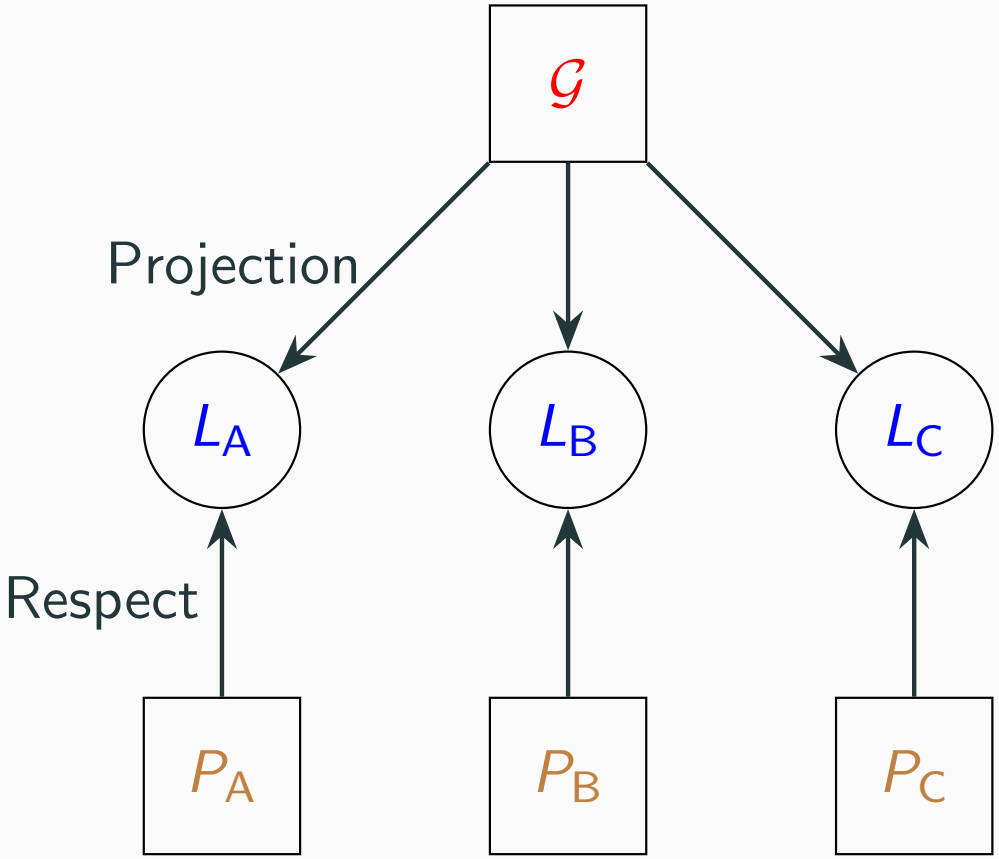
The communication system is seen as a whole.

Local



An actor's individual perspective.

Top-Down Approach



1. **Global view**
2. **Local view**
3. **Processes**

Top-Down Approach

Steps:

1. Write the global specification
2. Project to obtain the local specifications
3. Write the local programs that respect the specifications

Problem: difficult to include existing code and architecture.

Aim

Problem:

Debug and understand legacy code

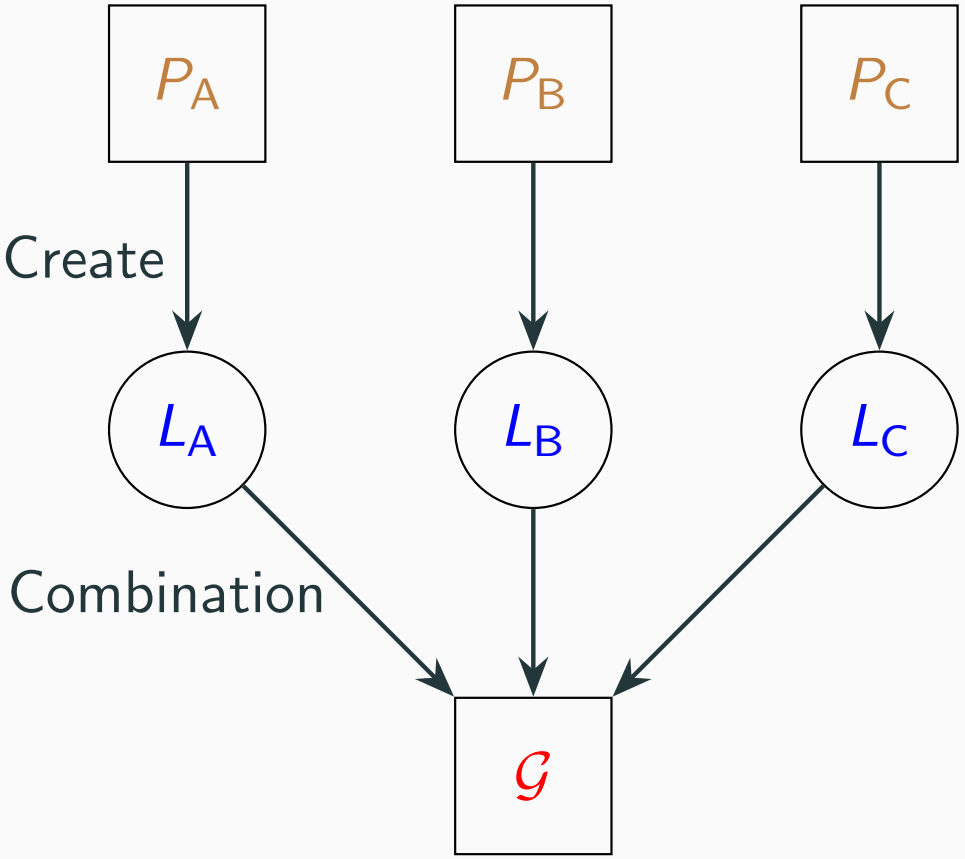
How?

Extract global specification from code

Support

Tool based on statical analysis

Bottom-up Approach



1. **Processes**
2. **Local view**
3. **Global view**

Bottom-up approach

Extraction steps:

1. Analyze an input source code
2. Create the local views
3. Combine local views to create an approximated global view

Over-approximation: explore all possible behaviors (including some that may not occur).

Why

Benefits

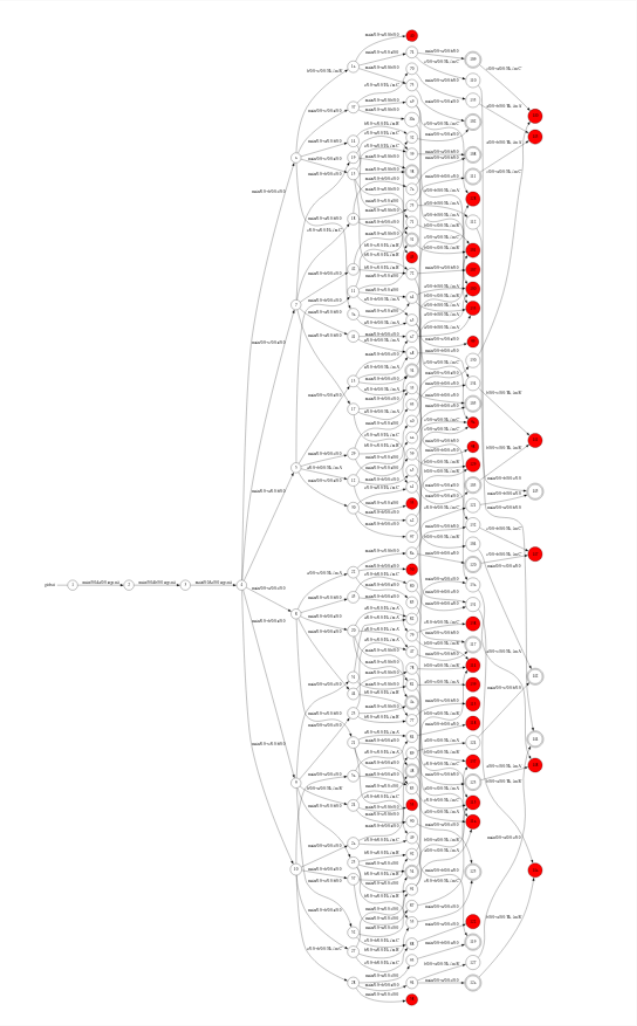
- Improves understanding of the code
  - Give also good behavior
- Highlight bugs
  - Like deadlock, race condition, etc...

Requirements

- Automatic extraction
- Target mainstream languages
- Support creation and removal of participants
- Capture good behaviors and highlight misbehavior
- Use a simple notation as output

Problems

- **Undecidability**: it would imply deciding termination
  - Use of over-approximation
- **Huge descriptions**: by nature of distributed systems
  - Needs of abstraction mechanism



Motivating Examples

Two examples:

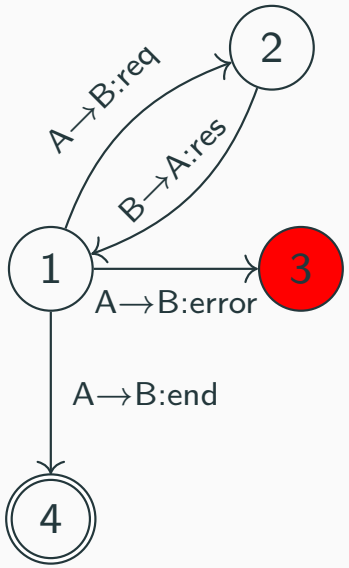
Dining Philosophers
(deadlock)

Bank Account
(race condition)

Deadlock

Final state: all the active participant are in a *local* final state.

Deadlock: it can occur if a non-final state without outgoing transitions is present.



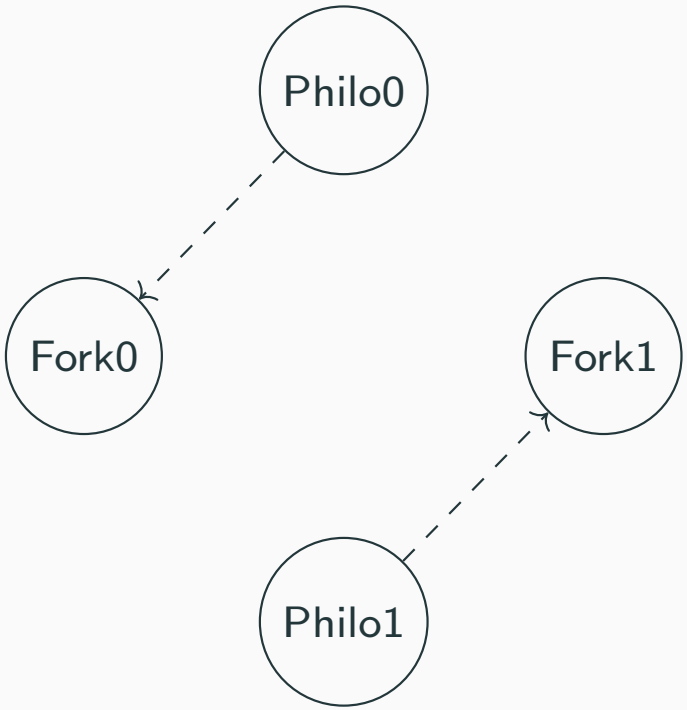
Dining Philosophers example

```
philosopher(Fork1, Fork2) →
  send req to Fork1,
  receive ack from Fork1,
  send req to Fork2,
  receive ack from Fork2,
  eat(),
  send release to Fork1,
  send release to Fork2,
  philosopher(Fork1, Fork2).

fork() →
  receive req from Phil,
  send ack to Phil,
  receive release from Phil,
  fork().
```

Participants:

- 2 philosophers
- 2 forks



Dining Philosophers example's local views

Attention: merge of redundant communication!

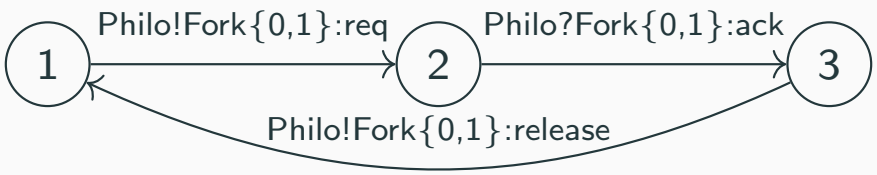


Figure 1: Philosopher's local view.

Figure 2: Fork's local view.

Dining Philosophers global-view

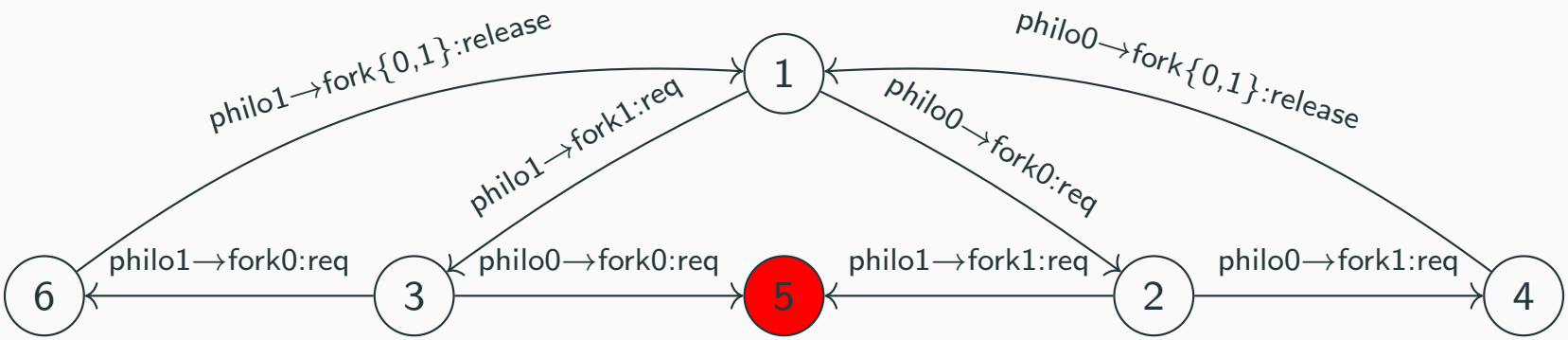


Figure 3: Global view of the Dining Philosophers example.

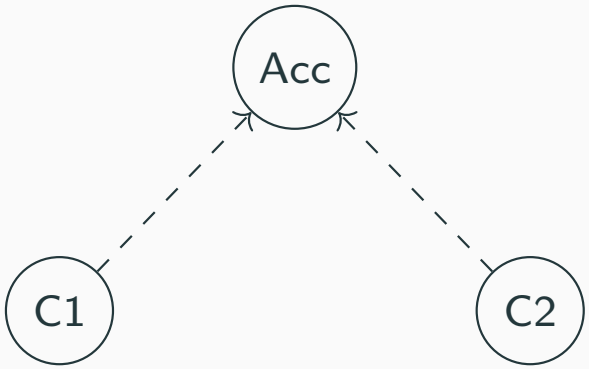
Bank Account example

```
account(Value) →
  receive
    read from Client →
        send Value to Client,
        account(Value);
    NewValue from Client →
      account(NewValue).

client() →
  send read to Acc,
  receive Value from Acc,
  % operations on Value
  send NewValue to Acc.
```

Participants:

- 1 account
- 2 clients



Bank Account example's local views

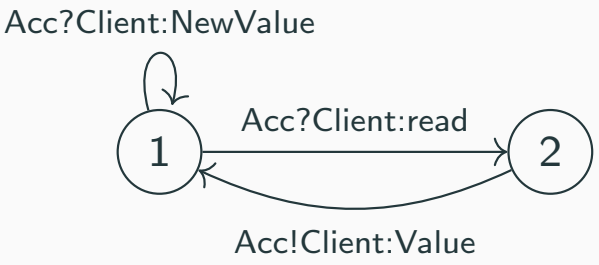


Figure 4: Account's local view.

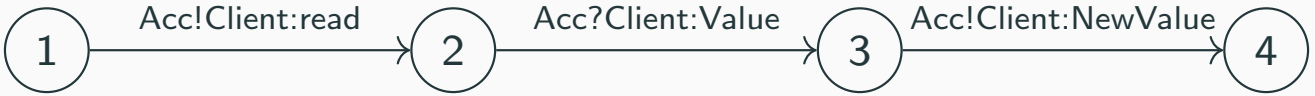


Figure 5: Clients' local view.

Bank Account global-view

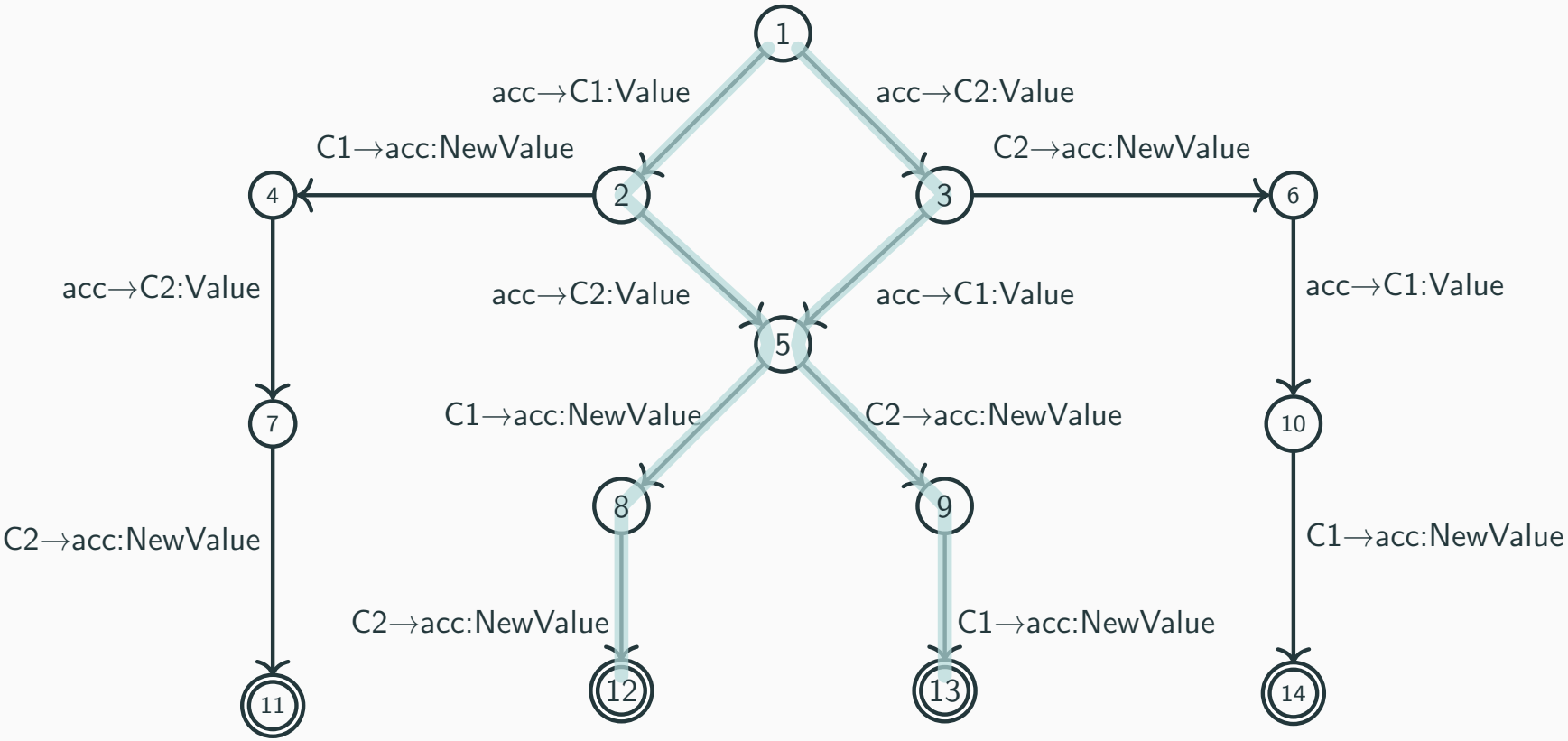


Figure 6: Global view of the Bank Account example

Conclusion

Summary

- Problem: difficult to express legacy code
- Bridge the gap between programming and theoretical frameworks
- Use of bottom-up and over-approx techniques

Future work: create the tool as described (WIP on `github.com/gabrielegenovese/chorer`) and evaluate it on real-world examples.

Thanks for listening!